UART FULL-DUPLEX

Name : Gautam Raj Chhabra
Project: UART Full-Duplex
Language: SystemVerilog
Tool Used: Vivado Xilinx

1. Abstract

This project implements a UART (Universal Asynchronous Receiver Transmitter) using SystemVerilog. The design supports full-duplex communication with both transmitter and receiver modules. The receiver uses 16x oversampling, mid-bit sampling, and a 2-FF synchronizer to ensure reliable data reception. The design is verified using a randomized testbench, assertions, and waveform analysis. RTL, simulation, and implementation are performed using Vivado.

2. Introduction

UART is a serial communication protocol widely used for asynchronous data transfer between devices.
In this project, a complete UART system is designed including:

- Transmitter (TX)
- Receiver (RX)
- Baud rate generator
- Interface-based communication
- Verification using SystemVerilog

The transmitter sends data serially, while the receiver reconstructs it using timing-based sampling.

3. Operational Conditions

- **IDLE**: Line remains high (tx = 1)
- **START BIT**: Transmission begins with a low signal (0)
- **DATA BITS**: 8-bit data transmitted LSB first
- **STOP BIT**: Line returns high (1)
- **VALID SIGNALS**:
    - valid: Indicates data is ready to transmit
    - ready: TX is ready to accept data
    - valid_rx: RX has received valid data

4. Design Features

- Full-duplex UART (TX + RX)
- SystemVerilog interface with modports
- 16x oversampling in receiver
- Mid-bit sampling for accuracy
- Falling edge detection for start bit
- 2-FF synchronizer for metastability handling
- Assertion-based verification
- Randomized testbench

5. Design Code

The UART design is implemented in SystemVerilog using modular architecture. It consists of separate transmitter, receiver, baud rate generator, and interface modules. The transmitter uses a finite state machine to serialize data, while the receiver reconstructs data using oversampling and mid-bit sampling techniques. Proper synchronization and control logic ensure accurate and reliable communication.

```
interface  uart_if;
  logic clk;
  logic reset;
  logic baud_tick;
  logic baud_tick_16;

  logic [7:0]data_in;
  logic valid;
  logic tx;
  logic ready;

  logic rx;
  logic [7:0]data_out;
  logic valid_rx;


  modport baud_gen_mp(
    input clk, reset,
    output baud_tick, baud_tick_16);

  modport uart_tx_mp(
```

```
    input clk, reset, valid, baud_tick, data_in,
    output tx, ready);

  modport uart_rx_mp(
    input clk, reset, baud_tick_16, rx,
    output data_out, valid_rx);

endinterface


// baud_generator

module baud_gen(uart_if.baud_gen_mp intf);
  localparam BAUD_DIV= 50;
  localparam BAUD_DIV_16= 50/16;

  logic [$clog2(BAUD_DIV)-1 :0]count;
   logic [$clog2(BAUD_DIV_16)-1:0] count16;

  always_ff@(posedge intf.clk or posedge intf.reset)
  begin

    if(intf.reset)
      begin
      count<=0;
      count16<=0;
    intf.baud_tick<=0;
    intf.baud_tick_16<=0;
      end

    else
      begin
        if(count==BAUD_DIV-1)
          begin
          count<=0;
        intf.baud_tick<=1;
          end
        else
          begin
```

```
            count<=count+1;
            intf.baud_tick<=0;
          end

          if (count16 == BAUD_DIV_16-1) begin
        count16 <= 0;
        intf.baud_tick_16 <= 1;
      end else begin
        count16 <= count16 + 1;
        intf.baud_tick_16 <= 0;
      end
      end

  end
endmodule


// TRANSMITTER

module uart_tx(uart_if.uart_tx_mp intf);

  typedef enum logic [1:0] {IDLE, START, DATA, STOP}
state_t;
  state_t state, next_state;

  logic [7:0] shift_reg;
  logic [2:0] bit_count;

  // state register
  always_ff @(posedge intf.clk or posedge intf.reset)
begin
    if (intf.reset)
      state <= IDLE;
    else
      state <= next_state;
  end

  // next state logic
  always_comb begin
```

```
    next_state = state;

    case (state)
      IDLE:
        if (intf.valid && intf.ready)
          next_state = START;

      START:
        if (intf.baud_tick)
          next_state = DATA;

      DATA:
        if (intf.baud_tick && bit_count == 7)
          next_state = STOP;

      STOP:
        if (intf.baud_tick)
          next_state = IDLE;

      default:
        next_state = IDLE;
    endcase
  end

  // output logic
  always_ff @(posedge intf.clk or posedge intf.reset)
begin
    if (intf.reset) begin
      intf.tx    <= 1;
      intf.ready <= 1;
      shift_reg  <= 0;
      bit_count  <= 0;
    end
    else begin
      case (state)

        IDLE: begin
          intf.tx    <= 1;
          intf.ready <= 1;
```

```
          if (intf.valid && intf.ready) begin
            shift_reg <= intf.data_in;
            bit_count <= 0;
            intf.ready <= 0;
          end
        end

        START: begin
          if (intf.baud_tick)
            intf.tx <= 0;
        end

        DATA: begin
          if (intf.baud_tick) begin
            intf.tx     <= shift_reg[0];
            shift_reg   <= {1'b0, shift_reg[7:1]};
            bit_count   <= bit_count + 1;
          end
        end

        STOP: begin
          if (intf.baud_tick) begin
            intf.tx    <= 1;
            intf.ready <= 1;
          end
        end

      endcase
    end
  end

  property tx_handshake;
  @(posedge intf.clk)
  (intf.valid && !intf.ready) |-> $stable(intf.data_in);
endproperty

assert property(tx_handshake)
  else $error("Data changed while TX not ready");
```

```
endmodule

module uart_rx(uart_if.uart_rx_mp intf);

  typedef enum logic [1:0] {IDLE, START, DATA, STOP}
state_t;
  state_t state, next_state;

  logic [7:0] shift_reg;
  logic [2:0] bit_count;
  logic [3:0] sample_count;

  logic rx_sync1, rx_sync2;
  logic rx_d;


  always_ff @(posedge intf.clk or posedge intf.reset)
begin
    if (intf.reset) begin
      rx_sync1 <= 1;
      rx_sync2 <= 1;
      rx_d     <= 1;
    end else begin
      rx_sync1 <= intf.rx;
      rx_sync2 <= rx_sync1;
      rx_d     <= rx_sync2;
    end
  end


  always_ff @(posedge intf.clk or posedge intf.reset)
begin
    if (intf.reset)
      state <= IDLE;
    else
      state <= next_state;
  end

  always_comb begin
```

```
    next_state = state;

    case (state)

      IDLE:
        if (rx_d == 1 && rx_sync2 == 0)
          next_state = START;

      START:
        if (intf.baud_tick_16 && sample_count == 7) begin
          if (rx_sync2 == 0)
            next_state = DATA;
          else
            next_state = IDLE;
        end

      DATA:
        if (intf.baud_tick_16 && sample_count == 15 &&
bit_count == 7)
          next_state = STOP;

      STOP:
        if (intf.baud_tick_16 && sample_count == 15)
          next_state = IDLE;

    endcase
  end


  always_ff @(posedge intf.clk or posedge intf.reset)
begin
    if (intf.reset) begin
      shift_reg     <= 0;
      bit_count     <= 0;
      sample_count  <= 0;
      intf.data_out <= 0;
      intf.valid_rx <= 0;
    end
    else begin
```

```
case (state)

  IDLE: begin
    bit_count <= 0;
    sample_count <= 0;

    if (rx_d == 1 && rx_sync2 == 0)
      intf.valid_rx <= 0;
  end

  START: begin
    if (intf.baud_tick_16) begin
      if (sample_count == 7)
        sample_count <= 0;
      else
        sample_count <= sample_count + 1;
    end
  end

  DATA: begin
    if (intf.baud_tick_16) begin
      if (sample_count == 15) begin
        sample_count <= 0;

        shift_reg <= {rx_sync2, shift_reg[7:1]};
        bit_count <= bit_count + 1;
      end
      else begin
        sample_count <= sample_count + 1;
      end
    end
  end

  STOP: begin
    if (intf.baud_tick_16) begin
      if (sample_count == 15) begin
        intf.data_out <= shift_reg;
        intf.valid_rx <= 1;
```

```
            sample_count <= 0;
          end
          else begin
            sample_count <= sample_count + 1;
          end
        end
      end

    endcase
  end
end

property rx_data_stable;
  @(posedge intf.clk)
  intf.valid_rx |-> $stable(intf.data_out);
endproperty

assert property(rx_data_stable)
  else $error("RX data changed while valid_rx is high");

endmodule
```

6.Testbench

The testbench is developed using SystemVerilog and includes class-based randomization to generate test data. A loopback mechanism (TX → RX) is used to verify end-to-end functionality. The testbench applies stimulus in a clock-synchronized manner and checks correctness using valid_rx signaling and display outputs. Assertions are also used to validate protocol behavior.

```
`timescale 1ns/1ps

module testbench;

  uart_if intf();

  baud_gen bg(intf);
  uart_tx  tx(intf);
  uart_rx  rx(intf);
```

```
  assign intf.rx = intf.tx;

  class packet;
    rand bit [7:0] data;
  endclass

  packet pkt;

  initial begin
    intf.clk = 0;
    forever #5 intf.clk = ~intf.clk;
  end

  initial begin
    $monitor("t=%0t valid=%0d ready=%0d tx=%0d rx=%0d
valid_rx=%0d data_out=%h",
              $time, intf.valid, intf.ready, intf.tx,
intf.rx, intf.valid_rx, intf.data_out);
  end

  initial begin

    pkt = new();

    intf.reset   = 1;
    intf.valid   = 0;
    intf.data_in = 0;

    #20;
    intf.reset = 0;

    repeat (5) begin

      wait(intf.ready);

      @(posedge intf.clk);

      assert(pkt.randomize());
      intf.data_in = pkt.data;
```

```
      intf.valid   = 1;

      @(posedge intf.clk);
      intf.valid = 0;

while (intf.valid_rx != 1)
  @(posedge intf.clk);

$display(" ");
$display("Sent Data     = %h", pkt.data);
$display("Received Data = %h", intf.data_out);
$display(" ");
      #1000;

    end

    #100000;
    $finish;

  end

endmodule
```

7. Simulation Result

The UART design is simulated to verify correct transmission and reception.

- Data transmitted is correctly received
- valid_rx indicates correct sampling
- No data corruption observed

Example:- Sent Data = 67
Received Data = 67

Sent Data = 22
Received Data = 22

Sent Data = dc
Received Data = dc

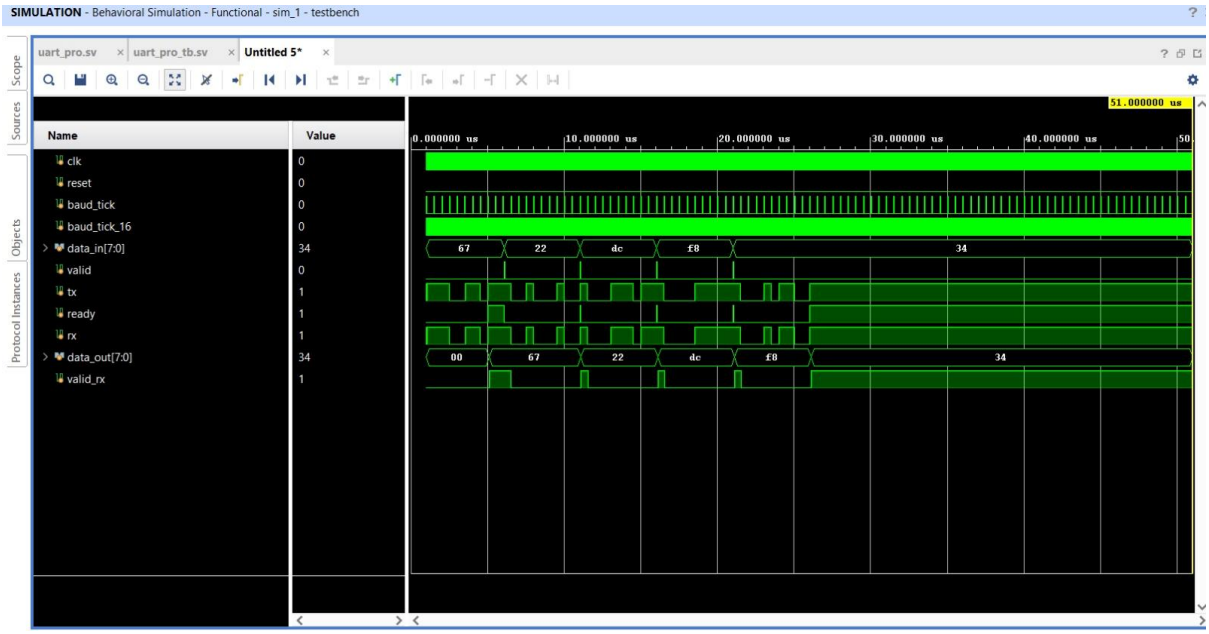
8. Waveform

The waveform demonstrates correct UART behavior:

- Start bit detection using falling edge
- Data bits sampled at mid-bit position
- Stop bit validation
- valid_rx asserted after full frame reception

Observations:

- Data is correctly aligned with baud timing
- Oversampling ensures accurate sampling
- No glitches or metastability issues observed

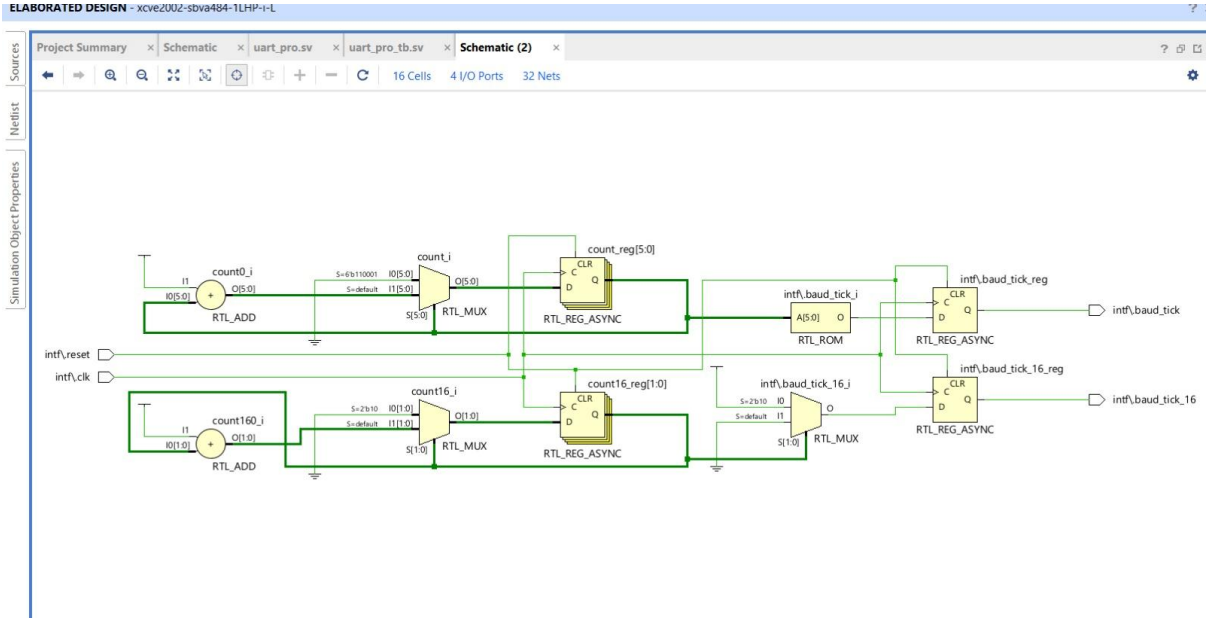


9.RTL Schematic

The RTL schematic includes:

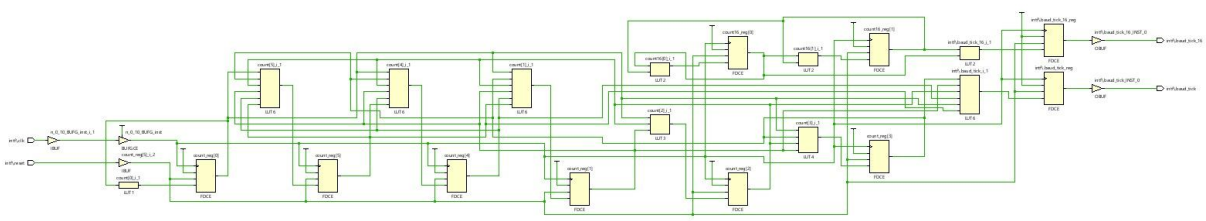
- FSM for TX and RX
- Shift registers
- Counters for bit and sample tracking
- Synchronizer logic

This confirms correct structural design



10. Synthesis Schematic

The synthesized design was analyzed using Vivado to verify the hardware-level implementation. The schematic shows the structural mapping of the UART modules, including state machines, shift registers, and control logic. It confirms that the RTL design is correctly translated into combinational and sequential elements without any unintended latches. The design is fully synthesizable and maintains proper signal connectivity across transmitter and receiver blocks



11. Why This Design is Robust

This project was improved significantly after facing real simulation issues.
Problems Faced:

- Incorrect waveform output (data_out = 00)
- Misalignment between TX and RX timing
- Sampling errors due to improper bit timing

- Start bit detection instability
- Metastability risk in asynchronous RX input

Solutions Implemented:

1. Oversampling (16x)

- Receiver samples input 16 times per bit
- Improves timing accuracy
- Helps recover data even with slight timing mismatch

---

2. Mid-Bit Sampling

- Data sampled at center of bit (sample_count = 7 or 15)
- Avoids edge noise and transitions
- Ensures stable data capture

---

3. Falling Edge Detection

- Detects start bit using: rx_d == 1 && rx == 0
- Prevents false triggering

---

4. 2-FF Synchronizer

- Protects against metastability
- Ensures safe clock-domain crossing

---

5. Proper Timing Verification

- Learned that: Data must be read only when valid_rx = 1
- Early sampling caused incorrect outputs

12. Conclusion

The UART system is successfully implemented and verified.

- Accurate data transmission and reception achieved
- Robust receiver design using oversampling and synchronization
- Verified using randomized testbench and assertions
- Demonstrates strong understanding of timing-critical digital design